

Gerenciamento de Estado e Ciclo de Vida dos Componentes

Sumário:

1. Gerenciamento de Estado em Componentes React.....	2
1.1 Gerenciamento de Estado em Componentes de Classe.....	2
1.2 Gerenciamento de Estado em Componentes Funcionais.....	8
2. Gerenciamento do Ciclo de Vida dos Componentes React.....	15
2.1 Ciclo de Vida em Componentes de Classe.....	15
2.2 Ciclo de Vida em Componentes Funcionais.....	17
3. Gerenciamento de eventos no React.....	23
3.1 Adicionando Event Handlers.....	23
3.2 Passando Argumentos para Event Handlers.....	23
3.3 Eventos Mais Comuns no React.....	24
3.4 Prevenindo o Comportamento Padrão.....	25
4. Comunicando componentes.....	26
4.1 Comunicando de Pai para Filho (via props).....	26
4.2 Comunicando de Filho para Pai (via props e callbacks).....	27
4.3 Comunicando entre Componentes Irmãos (via Pai).....	28
Referências.....	30

1. Gerenciamento de Estado em Componentes React

O gerenciamento de estado é um conceito essencial em React, pois permite que componentes armazenem informações dinâmicas e retenham dados que mudam com o tempo. Existem duas abordagens principais para gerenciar o estado em React: em **componentes de classe** e em **componentes funcionais**.

1.1 Gerenciamento de Estado em Componentes de Classe

Nos componentes de classe em React, o estado é uma estrutura que armazena informações dinâmicas e controladas pelo componente, permitindo que ele responda a eventos, como cliques de botões ou dados que mudam ao longo do tempo. O estado é definido no construtor da classe usando **this.state**, e ele deve ser um objeto que contém todas as propriedades de estado que o componente precisa. Assim, o estado é mantido internamente no componente e pode ser alterado com o método **this.setState()**, o que dispara uma nova renderização do componente para refletir as mudanças. Neste contexto, observamos que o estado do componente não é uma simples variável, ele é gerenciado pelo React que determina a necessidade de cada componente ser renderizado novamente.

1.1.1 Atualizando o Estado com um Objeto

A maneira mais comum de usar o método **setState()** é passando um objeto com as propriedades do estado que você deseja atualizar. O React irá mesclar automaticamente essas propriedades com o estado atual. Veja abaixo sua sintaxe:

```
this.setState({  
  chave: novoValor  
});
```

Veja um exemplo de atualização de estado usando **setState()**:

```
this.setState({  
  contador: this.state.contador + 1  
});
```

Neste exemplo, o valor do estado **contador** é incrementado em **1**, e o React renderiza o componente para refletir a mudança no navegador.

Vejamos um exemplo mais completo. No quadro abaixo, definimos um componente de classe em React chamado **Contador**, que gerencia um estado interno e o atualiza de forma dinâmica quando o usuário interage com a interface.

```
import { Component } from "react";
class Contador extends Component {
  constructor(props) {
    super(props);
    this.state = { // Definindo o estado inicial no construtor
      contador: 0,
    };
  }
  render() {
    return (
      <div>
        <p>Contador: {this.state.contador}</p>
        <button onClick={() => this.incrementar()}>Incrementar</button>
      </div>
    );
  }
  // Método para atualizar o estado
  incrementar = () => {
    // Atualiza o estado com setState
    this.setState({ contador: this.state.contador + 1 });
  }
}
export default Contador;
```

Veja como funciona a definição e atualização do estado em componentes de classe:

- **Definição do estado:** O estado inicial é definido dentro do construtor da classe com **this.state = { contador: 0 }**.
- **Acesso ao estado:** O estado é acessado dentro do componente via **this.state.nomeDoEstado**. No exemplo, usamos **this.state.contador** para exibir o valor do contador.
- **Atualização do estado:** Para alterar o estado, usamos o método **this.setState()**. Isso é crucial, pois o React precisa ser notificado sobre mudanças no estado para que possa renderizar o componente e atualizar a interface do usuário. No

exemplo, a função **incrementar()** chama **this.setState({ contador: this.state.contador + 1 })** para incrementar o valor do contador.

Veja na figura abaixo o resultado no navegador quando clicamos 1 vez no botão “Incrementar”:

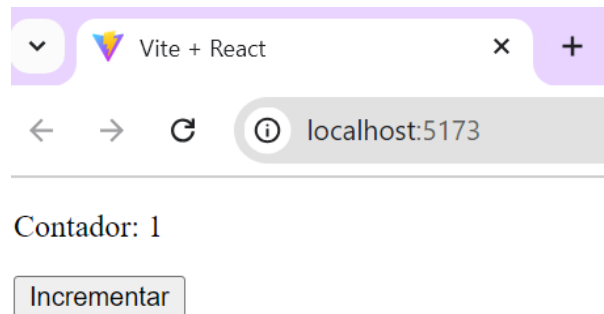


Figura 1: Exemplo de um funcionamento de estado em componente de classe

Descrição: A imagem mostra uma página web no navegador, com o título "Vite + React", rodando em "localhost:5173". O conteúdo da página exibe um texto que diz "Contador: 1", seguido por um botão abaixo com o rótulo "Incrementar".

1.1.2 Atualizando o Estado com uma Função

Em alguns casos, o estado pode depender do valor anterior (como ao incrementar contadores ou manipular dados dinâmicos). Como o **setState** é **assíncrono**, há um risco de que múltiplas atualizações possam ser aplicadas incorretamente se você tentar usar diretamente o valor de **this.state**. Para evitar isso, você pode usar uma **função de atualização** no **setState** que recebe o **estado anterior** como argumento. Veja abaixo sua sintaxe:

```
this.setState((prevState, props) => {  
  return {  
    chave: novoValor  
  };  
});
```

Onde:

- **prevState:** Representa o estado anterior do componente.
- **props:** Representa as **props** do componente no momento da atualização (geralmente opcional).

Em nosso exemplo com o componente **Contador**, podemos atualizar o método **incrementar()** da seguinte forma:

```
// Método para atualizar o estado
incrementar = () => {
  // Atualiza o estado com setState
  this.setState((prevState) => ({
    contador: prevState.contador + 1
  }));
}
```

Neste exemplo:

- A função recebe **prevState** como argumento, garantindo que a atualização seja baseada no valor mais recente do estado.
- Isso é útil para evitar problemas quando várias atualizações de estado ocorrem ao mesmo tempo.

1.1.3 O método setState é Assíncrono

É importante entender que o **setState** é **assíncrono**, ou seja, o React pode agrupar várias atualizações de **estado** para otimizar o desempenho. Isso significa que acessar **this.state** logo após chamar **setState** pode não refletir o estado atualizado.

Por exemplo, este código pode não funcionar conforme o esperado:

```
this.setState({ contador: this.state.contador + 1 });
console.log(this.state.contador); // Pode exibir o valor antigo
```

Para garantir que você esteja usando o valor atualizado do estado, use a forma de função mencionada anteriormente ou use a opção de **callback** do método **setState** (que veremos a seguir).

1.1.4 Função de Callback no setState

Se você quiser executar código imediatamente após a atualização do estado ser aplicada e o componente ser renderizado, pode passar uma **função de callback** como segundo argumento do **setState**.

Veja abaixo sua sintaxe:

```
this.setState({chave: novoValor},() => {  
    // Código executado após o estado ser atualizado  
})  
);
```

Em nosso exemplo com o componente **Contador**, podemos atualizar o método **incrementar()** da seguinte forma:

```
// Método para atualizar o estado  
incrementar = () => {  
    // Atualiza o estado com setState  
    this.setState({ contador: this.state.contador + 1 }, () => {  
        console.log("Estado atualizado:", this.state.contador);  
    });  
}
```

Veja na figura abaixo o resultado no navegador quando clicamos 1 vez no botão "Incrementar":

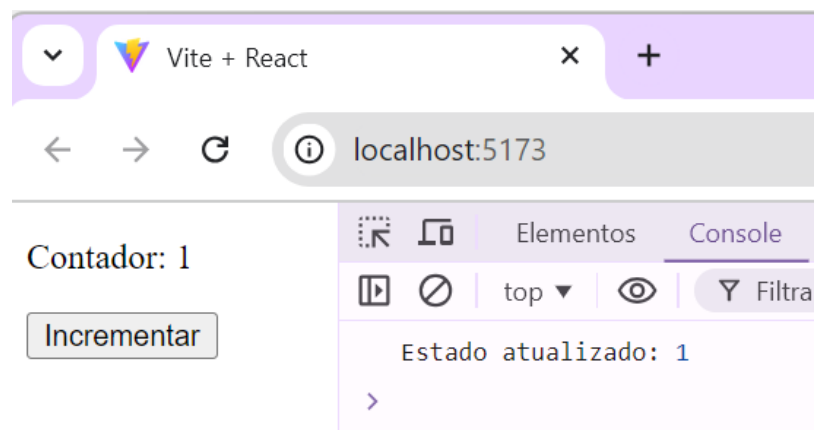


Figura 2: Exemplo de um funcionamento da função de Callback no setState

Descrição: A imagem mostra uma página web no navegador, com o título "Vite + React", rodando em "localhost:5173". À esquerda, o conteúdo exibe um contador com o texto "Contador: 1" e um botão abaixo chamado "Incrementar". À direita, o console de desenvolvedor está aberto, mostrando a mensagem "Estado atualizado: 1".

1.1.5 Mesclagem do Estado com o setState

Ao contrário de substituir o estado inteiro, o **setState** mescla as propriedades do estado fornecidas com o estado existente. Isso significa que você pode atualizar uma ou algumas propriedades do estado sem perder o restante.

Veja no quadro abaixo um exemplo de estado com múltiplas propriedades. Neste exemplo, criamos o componente **Perfil** contendo três propriedades: nome, idade e cidade.

```
import { Component } from "react";
class Perfil extends Component {
  constructor(props) {
    super(props);
    this.state = {
      nome: "Maria",
      idade: 25,
      cidade: "São Paulo",
    };
  }
  atualizarIdade = () => {
    // Atualiza apenas a idade, mantendo as outras propriedades do estado
    this.setState({
      idade: 26,
    });
  };
  render() {
    return (
      <div>
        <p>Nome: {this.state.nome}</p>
        <p>Idade: {this.state.idade}</p>
        <p>Cidade: {this.state.cidade}</p>
        <button onClick={this.atualizarIdade}>Atualizar Idade</button>
      </div>
    );
  }
}
export default Perfil;
```

Neste exemplo:

- O estado inicial tem três propriedades: **nome**, **idade** e **cidade**.

- Ao chamar **this.setState({ idade: 26 })**, apenas a propriedade **idade** é atualizada. As outras propriedades (**nome** e **cidade**) permanecem inalteradas.

Veja na figura abaixo o resultado no navegador quando clicamos no botão “Atualizar Idade”:

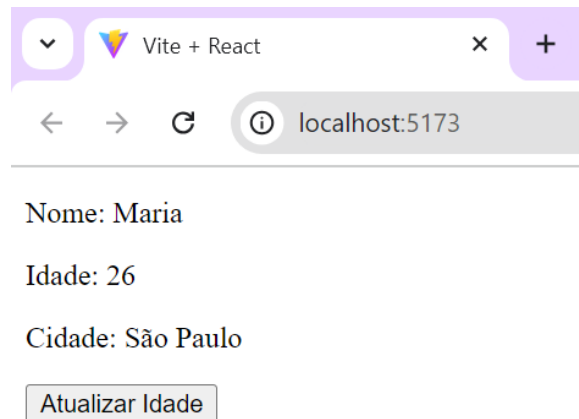


Figura 3: Exemplo de um funcionamento da mesclagem do estado com o método `setState`

Descrição: A imagem mostra uma página web no navegador, com o título "Vite + React", rodando em "localhost:5173". O conteúdo da página exibe as informações: "Nome: Maria", "Idade: 26" e "Cidade: São Paulo". Abaixo desses textos, há um botão com o rótulo "Atualizar Idade".

1.2 Gerenciamento de Estado em Componentes Funcionais

Em componentes funcionais, o gerenciamento de estado em React é feito usando os [Hooks](#). Os Hooks foram introduzidos no React 16.8 para permitir que componentes funcionais gerenciem estado e ciclo de vida de forma simples e eficaz, sem precisar converter esses componentes em classes.

Veremos nesta seção o Hook **useState**, muito utilizado para gerenciar o estado de componentes funcionais. O **useState** é o Hook mais básico e amplamente usado para gerenciamento de estado local em componentes funcionais. Ele permite criar variáveis de estado e atualizá-las conforme necessário, assim como o método **this.setState()** em componentes de classe.

O Hook **useState** retorna um array com dois valores:

- O valor atual do estado.
- Uma função para atualizar esse valor.

Veja abaixo sua sintaxe:

```
const [estado, setEstado] = useState(valorInicial);
```

Onde:

- **estado:** O valor do estado atual.
- **setEstado:** Função para atualizar o valor do estado.
- **valorInicial:** O valor inicial que o estado terá quando o componente for montado.

Veja no quadro abaixo como criar um simples contador que utiliza o Hook **useState** para gerenciar o estado local:

```
import { useState } from 'react';
function Contador() {
  // Declaramos uma variável de estado chamada 'contador' com valor inicial 0
  const [contador, setContador] = useState(0);
  return (
    <div>
      <p>Contador: {contador}</p>
      <button onClick={() => setContador(contador + 1)}>Incrementar</button>
    </div>
  );
}
export default Contador;
```

Neste exemplo:

- **Importação do useState:** Importamos o Hook **useState** de React, que permite gerenciar o estado em um componente funcional.
- **Declaração do Estado:** Dentro do componente, **const [contador, setContador] = useState(0)** define uma variável de estado chamada **contador** com valor inicial **0**, e a função **setContador** para atualizar esse valor.
- **Atualizando o Estado:** A função **setContador** é usada para atualizar o valor de contador. Assim, quando o botão é clicado, chamamos **setContador(contador + 1)**, o que atualiza o estado de contador com o novo valor.

- **Renderização Automática:** O React detecta a mudança de estado e renderiza o componente para exibir o novo valor do contador, garantindo que a interface reflita as alterações no estado.

Veja abaixo o resultado no navegador quando clicamos 1 vez no botão “Incrementar”:

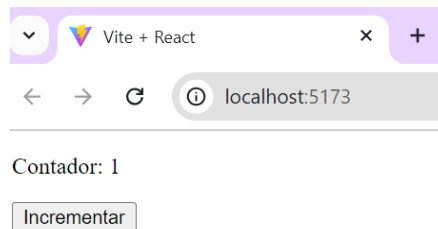


Figura 4: Exemplo de um funcionamento de estado em componente funcional

Descrição: A imagem mostra uma página web no navegador, rodando em "localhost:5173". O conteúdo da página exibe um texto que diz "Contador: 1", seguido por um botão abaixo com o rótulo "Incrementar".

1.2.1 Atualizando Estado com base no Valor Anterior

Quando o novo valor do estado depende do estado anterior, você deve usar a **função de atualização** do **useState**. Isso evita erros, especialmente quando múltiplas atualizações de estado ocorrem em sequência. Veja abaixo um contador com função de atualização do **useState**:

```
import { useState } from "react";
function Contador() {
  const [contador, setContador] = useState(0);
  return (
    <div>
      <p>Contador: {contador}</p>
      <button onClick={() => setContador((prevContador) => prevContador + 1)}>
        Incrementar
      </button>
    </div>
  );
}
export default Contador;
```

Neste exemplo, **setContador** recebe uma função que usa o valor anterior (**prevContador**). Isso garante que a atualização seja sempre baseada no estado mais recente, evitando problemas quando várias atualizações ocorrem rapidamente.

1.2.2 Estado Complexo

O **useState** também pode gerenciar **objetos** e **arrays** como estado. No entanto, ao atualizar estados complexos, você deve garantir que apenas as partes necessárias sejam atualizadas, sem sobrescrever as demais propriedades.

1.2.2.1 Estado com Objetos

Se o **estado for um objeto** contendo várias propriedades, você precisa garantir que apenas a propriedade alterada seja atualizada, enquanto as demais são mantidas. Para fazer isso, utilizamos o **spread operator (...)** para mesclar o novo valor com o estado anterior.

Veja no quadro abaixo um exemplo de código React que define um componente funcional chamado **PerfilUsuario** e que usa o Hook **useState** para gerenciar um estado que contém informações de um usuário, como nome, idade e cidade:

```
import { useState } from 'react';

function PerfilUsuario() {
  const [usuario, setUsuario] = useState({
    nome: 'João',
    idade: 25,
    cidade: 'São Paulo',
  });

  const atualizarCidade = () => {
    setUsuario((prevUsuario) => ({
      ...prevUsuario, // Preserva as outras propriedades do estado
      cidade: 'Rio de Janeiro', // Atualiza apenas a cidade
    }));
  };

  return (
    <div>
      <p>Nome: {usuario.nome}</p>
      <p>Idade: {usuario.idade}</p>
      <p>Cidade: {usuario.cidade}</p>
      <button onClick={atualizarCidade}>Mudar Cidade</button>
    </div>
  );
}

export default PerfilUsuario;
```

Neste exemplo:

- O estado inicial é um objeto com as propriedades **nome**, **idade** e **cidade**.
- O Hook **useState** retorna dois valores:
 - **usuario**: O valor atual do estado (um objeto com nome, idade e cidade).
 - **setUsuario**: A função que é usada para atualizar o estado.
- Para atualizar a **cidade**, utilizamos o **spread operator** (...). Isso mantém as propriedades **nome** e **idade** inalteradas enquanto atualiza apenas **cidade**.
- A função **setUsuario** recebe o estado anterior (**prevUsuario**) como argumento e retorna um novo objeto que contém todas as propriedades do estado anterior (**...prevUsuario**), mas com a propriedade **cidade** alterada para 'Rio de Janeiro'.

Veja abaixo o resultado no navegador quando clicamos no botão “Mudar Cidade”:

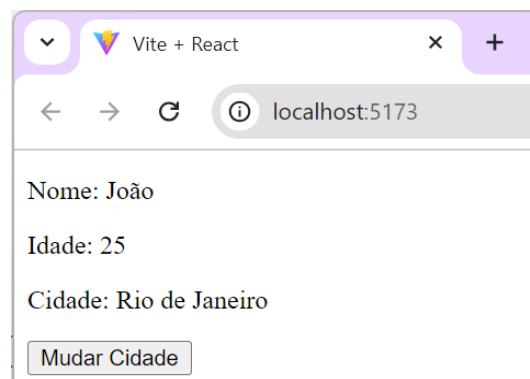


Figura 5: Exemplo de um funcionamento de estado com objetos

Descrição: A imagem mostra uma página web no navegador, com o título "Vite + React", rodando em "localhost:5173". O conteúdo da página exibe as informações: "Nome: João", "Idade: 25" e "Cidade: Rio de Janeiro".

1.2.2.2 Estado com Arrays

Para gerenciar estados que são arrays, você pode adicionar, remover ou modificar itens do array diretamente no estado. Como o **useState** não faz a atualização de forma imutável automaticamente, você precisa garantir que o array seja imutável, ou seja, criar um novo array ao invés de modificar o original.

Veja no quadro abaixo um exemplo:

```
import { useState } from 'react';
function ListaTarefas() {
  const [tarefas, setTarefas] = useState(['Estudar', 'Exercitar', 'Ler']);
  const adicionarTarefa = (novaTarefa) => {
    setTarefas([...tarefas, novaTarefa]); // Adiciona uma nova tarefa ao array
  };
  const removerTarefa = (index) => {
    setTarefas(tarefas.filter((_, i) => i !== index)); // Remove a tarefa pelo índice
  };
  return (
    <div>
      <ul>
        {tarefas.map((tarefa, index) => (
          <li key={index}>
            {tarefa}
            <button onClick={() => removerTarefa(index)}>Remover</button>
          </li>
        ))}
      </ul>
      <button onClick={() => adicionarTarefa('Nova Tarefa')}>Adicionar Tarefa</button>
    </div>
  );
}
export default ListaTarefas;
```

Neste exemplo:

- O **estado inicial** é um array de strings chamado tarefas.
- Para **adicionar um item**, usamos o **spread operator (...)** para criar um novo array contendo os itens anteriores e o novo item.
 - Veja que usamos **setTarefas([...tarefas, novaTarefa]);**
- Para **remover um item**, usamos o método **filter()** para criar um novo array sem o item a ser removido.
 - Veja que usamos **setTarefas(tarefas.filter((_, i) => i !== index));**
 - O método **filter()** cria um novo array sem o item cujo índice é igual ao valor de **index**. Isso garante que estamos trabalhando de forma imutável, sem modificar diretamente o array original.

OBS: O método **filter()** é uma função de array do JavaScript que cria um novo array contendo apenas os elementos que passam por um teste (função de callback). No exemplo acima, o teste é se o índice do elemento não corresponde ao índice que você deseja remover.

OBS2: A sintaxe do método **filter()** é a seguinte:

```
array.filter((elemento, index) => condição);
```

Onde: **elemento** é o valor do elemento atual no array; **index** é o índice do elemento atual; e **condição** é condição que deve ser verdadeira para o elemento ser mantido no novo array. Dessa forma, em **filter(, i) => i !== index**, o **_** é um placeholder para o valor do elemento, que não está sendo usado no código, já que a remoção é baseada no índice.

Veja abaixo o resultado no navegador quando clicamos no botão “Adicionar Tarefa”:

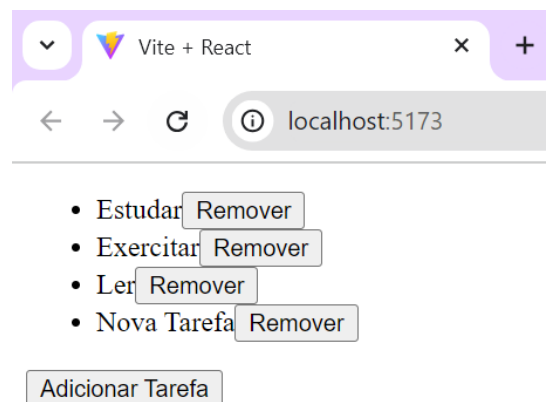


Figura 6: Exemplo de um funcionamento de estado com arrays

Descrição: A imagem mostra uma página web no navegador, com o título "Vite + React", rodando em "localhost:5173". O conteúdo da página exibe uma lista de tarefas, incluindo "Estudar", "Exercitar", "Ler" e "Nova Tarefa". Ao lado de cada tarefa, há um botão "Remover", que provavelmente permite excluir a tarefa correspondente. Abaixo da lista, há um botão "Adicionar Tarefa", que provavelmente adiciona uma nova tarefa à lista.

2. Gerenciamento do Ciclo de Vida dos Componentes React

O ciclo de vida dos componentes React refere-se às diferentes etapas pelas quais um componente React passa desde sua criação até sua remoção do DOM. Esses estágios incluem a montagem, atualização e desmontagem do componente. O React oferece métodos de ciclo de vida (para **componentes de classe**) e o Hook **useEffect** (para **componentes funcionais**) que permitem aos desenvolvedores interagir com esses momentos e executar ações específicas, como carregar dados, executar efeitos colaterais e limpar recursos.

2.1 Ciclo de Vida em Componentes de Classe

Nos **componentes de classe** do React, os métodos de ciclo de vida são **chamados automaticamente** durante as fases do ciclo de vida do componente. Essas fases podem ser classificadas em três grandes etapas:

- **Montagem:** Quando o componente é criado e inserido no DOM.
- **Atualização:** Quando o componente é re-renderizado devido à mudança de estado ou props.
- **Desmontagem:** Quando o componente é removido do DOM.

Considerando estas etapas, no quadro abaixo está um exemplo simples que utiliza os três principais métodos do ciclo de vida em componentes de classe: **componentDidMount**, **componentDidUpdate**, e **componentWillUnmount**. Esse exemplo simula o comportamento de um contador controlado por um botão, que também exibe mensagens no console quando o componente é montado, atualizado e desmontado.

```
import { Component } from "react";

class Contador extends Component {
  constructor(props) {
    super(props);
    // Estado inicial com o valor do contador
    this.state = {
      contador: 0,
    };
  }
  // Executado uma vez após o componente ser montado no DOM
  componentDidMount() {
    console.log("Componente montado! Contador inicial:", this.state.contador);
  }
}
```

```
// Executado sempre que o componente é atualizado (por exemplo, quando o estado muda)
componentDidUpdate(prevProps, prevState) {
  if (prevState.contador !== this.state.contador) {
    console.log(
      "Componente atualizado! Novo valor do contador:",
      this.state.contador
    );
  }
}
// Executado imediatamente antes de o componente ser desmontado do DOM
componentWillUnmount() {
  console.log("Componente será desmontado!");
}
// Método para incrementar o valor do contador
incrementar = () => {
  this.setState({ contador: this.state.contador + 1 });
};
render() {
  return (
    <div>
      <h1>Contador: {this.state.contador}</h1>
      <button onClick={this.incrementar}>Incrementar</button>
    </div>
  );
}
}
export default Contador;
```

Neste exemplo:

- **constructor(props):** Inicializa o estado do componente com o contador em 0.
- **componentDidMount():**
 - Esse método é chamado apenas uma vez, logo após o componente ser inserido no DOM. É um ótimo lugar para inicializar comportamentos, como fazer chamadas de API.
- **componentDidUpdate(prevProps, prevState):**
 - Esse método é chamado toda vez que o componente é atualizado, ou seja, quando as props ou o estado mudam.
 - O **componentDidUpdate** recebe como parâmetros o estado anterior (**prevState**) e as props anteriores (**prevProps**).
 - Aqui, ele verifica se o valor do contador foi alterado e, se foi, registra uma mensagem no console com o novo valor do contador.

- **componentWillUnmount():**

- Esse método é chamado imediatamente antes do componente ser removido do DOM. É usado principalmente para evitar vazamentos de memória.
- Aqui, ele apenas registra uma mensagem no console quando o componente estiver prestes a ser desmontado.

- **Método incrementar():**

- Este é o método responsável por incrementar o contador. Quando o botão "Incrementar" é clicado, ele chama **this.setState()** para atualizar o valor do contador no estado, o que aciona uma nova renderização e o método **componentDidUpdate()**.

Veja abaixo o resultado no navegador quando clicamos 2 vezes no botão "Incrementar":

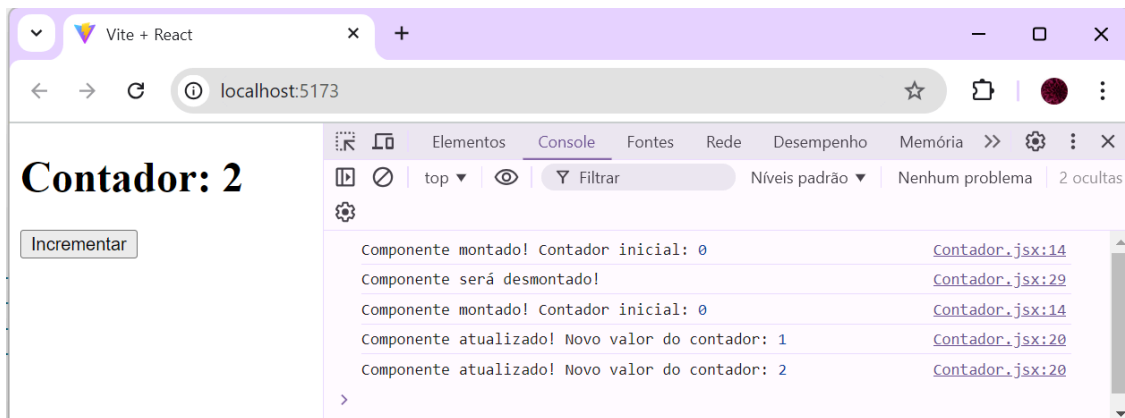


Figura 7: Exemplo do funcionamento do ciclo de vida em componentes de classe

Descrição: A imagem mostra uma página web com um contador exibindo o valor "2" e um botão "Incrementar". Ao lado, o console de desenvolvedor do navegador está aberto, mostrando mensagens sobre o ciclo de vida do componente React. O console indica que o componente foi montado com o contador inicial em "0", depois atualizado para "1" e "2", e também menciona que o componente será desmontado. Isso reflete o comportamento típico de um componente React durante sua montagem, atualização e desmontagem.

2.2 Ciclo de Vida em Componentes Funcionais

Em **componentes funcionais** no React, o gerenciamento do ciclo de vida dos componentes (montagem, atualização e desmontagem) é feito principalmente por meio do Hook **useEffect**. Enquanto nos componentes de classe o React fornece métodos de ciclo de vida como

componentDidMount, **componentDidUpdate**, e **componentWillUnmount**, nos componentes funcionais, o **useEffect** lida com esses mesmos ciclos, proporcionando uma forma mais flexível e simples de gerenciar efeitos colaterais.

OBS: No contexto do React, um efeito colateral refere-se a qualquer operação que afete algo fora do escopo de uma função ou componente puro. Isso inclui ações como:

- Fazer requisições de dados (API calls)
- Manipular diretamente o DOM (fora do React)
- Configurar timers ou intervalos (setInterval, setTimeout)
- Assinar ou cancelar eventos globais (como eventos de redimensionamento da janela)
- Acessar ou alterar o estado ou o ambiente do navegador (localStorage, cookies, etc.)

Veja abaixo a sintaxe básica do **useEffect**:

```
useEffect(() => {  
  // Efeito colateral (como buscar dados, manipular o DOM, etc.)  
  
  return () => {  
    // Função de limpeza (para desmontagem ou reset)  
  };  
}, [dependencias]);
```

Onde:

- **Primeiro argumento:** Uma função que define o efeito colateral a ser executado (pode ser buscar dados de uma API, manipular o DOM, etc.).
- **Segundo argumento:** Um array de dependências que controla quando o efeito será executado. O React compara os valores das dependências com a renderização anterior para decidir se o efeito deve ser executado novamente.

Veremos nesta seção como o **useEffect** pode ser usado nas fases de Montagem, Atualização e Desmontagem de um componente funcional.

Para executar um efeito apenas uma vez, na **montagem do componente** (como no **componentDidMount**), passamos um array de dependências vazio ([]) no **useEffect**. Isso garante que o efeito será executado somente na primeira renderização.

Veja um exemplo no quadro abaixo:

```
import { useEffect } from "react";
function ExemploMontagem() {
  useEffect(() => {
    console.log("O componente foi montado!");
    // Aqui você poderia fazer uma chamada de API ou configurar algo
  }, []); // O array vazio garante que o efeito seja executado apenas uma vez
  return (
    <div>
      <h1>Exemplo de Montagem</h1>
    </div>
  );
}
export default ExemploMontagem;
```

Onde:

- **useEffect(() => { ... }, [])**: A função **useEffect** é executada uma vez, quando o componente é montado. O array vazio indica que o efeito não depende de nenhuma variável e não será executado novamente.
- Neste exemplo, ele registra uma mensagem no console quando o componente é montado.

Veja abaixo o resultado no navegador:

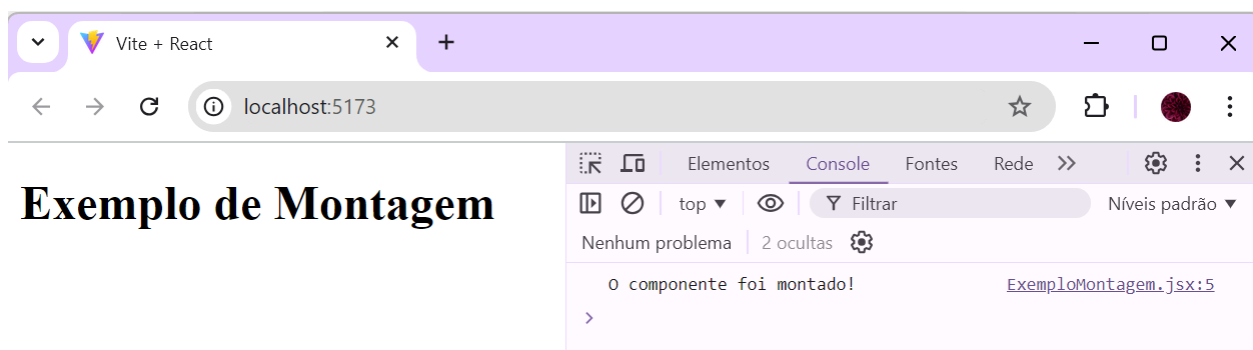


Figura 7: Exemplo do funcionamento do useEffect na montagem do componente

Descrição: A imagem mostra uma página web no navegador com o título "Vite + React" e o texto "Exemplo de Montagem" em destaque na página. Ao lado, o console de desenvolvedor está aberto e exibe a mensagem "O componente foi montado!".

OBS: Caso você observe no seu console que a mensagem "O componente foi montado!" aparece duas vezes, isso pode estar sendo influenciado pelo comportamento do

<React.StrictMode>. Esta é uma ferramenta do React projetada para ajudar os desenvolvedores a identificar problemas potenciais no código e garantir que as melhores práticas recomendadas pelo React estejam sendo seguidas. Ele faz isso ao executar verificações adicionais durante o desenvolvimento, o que pode incluir renderizações extras dos componentes. Uma dessas verificações é a renderização dupla (ou renderização duas vezes) dos componentes em modo de desenvolvimento. Essa renderização extra ajuda a detectar efeitos colaterais inesperados ou incorretos que possam ocorrer durante a montagem e atualização dos componentes. Cabe ressaltar que isso não afeta o comportamento da aplicação no ambiente de produção.

Neste próximo exemplo, o **useEffect** será executado sempre que o valor de contador mudar, na fase de **atualização do componente**, o que é semelhante ao **componentDidUpdate** em componentes de classe.

```
import { useState, useEffect } from "react";

function ExemploAtualizacao() {
  const [contador, setContador] = useState(0);
  useEffect(() => {
    console.log("O contador foi atualizado! Valor atual:", contador);
  }, [contador]); // O efeito é executado toda vez que o contador muda
  return (
    <div>
      <h1>Contador: {contador}</h1>
      <button onClick={() => setContador(contador + 1)}>Incrementar</button>
    </div>
  );
}

export default ExemploAtualizacao;
```

Neste exemplo:

- **useEffect(() => { ... }, [contador]):** O efeito é executado toda vez que o valor de **contador** muda. O **console.log** registra a atualização do contador.
- O componente renderiza um contador com um botão para incrementá-lo.

Veja abaixo o resultado no navegador quando clicamos 2 vezes no botão “Incrementar”:

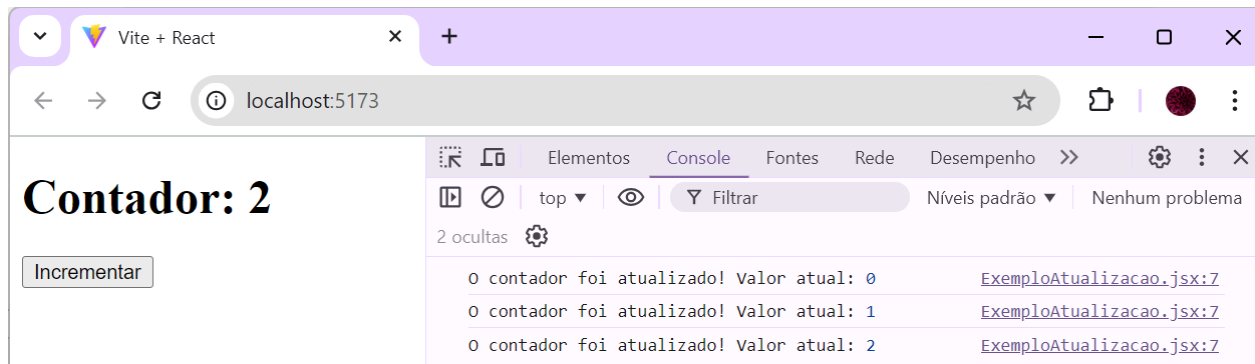


Figura 8: Exemplo do funcionamento do `useEffect` na atualização do componente

Descrição: A imagem mostra uma página web com um contador exibindo o valor "2" e um botão "Incrementar". Ao lado, o console de desenvolvedor do navegador está aberto e exibe mensagens indicando que o contador foi atualizado três vezes. As mensagens mostram os valores do contador: "0", "1" e "2", com a confirmação de que o valor atual foi atualizado. Isso reflete o comportamento de um componente React que está sendo renderizado a cada incremento no valor do contador.

Neste próximo exemplo, usamos o **`useEffect`** para configurar um temporizador (intervalo) quando o componente é montado e limpar esse intervalo quando o componente é **desmontado**. Isso é semelhante ao **`componentWillUnmount`**.

```
import { useState, useEffect } from "react";

function Temporizador() {
  const [segundos, setSegundos] = useState(0);

  useEffect(() => {
    const intervalo = setInterval(() => {
      setSegundos(s => s + 1);
    }, 1000);

    // Função de limpeza para desmontar o intervalo
    return () => {
      console.log("Componente desmontado! Limpando intervalo.");
      clearInterval(intervalo);
    };
  }, []); // O intervalo é configurado apenas na montagem

  return (
```

```
<div>
  <h1>Segundos: {segundos}</h1>
</div>
);
}
export default Temporizador;
```

Neste exemplo:

- **useEffect(() => { ... return () => { ... } }, [])**: O **useEffect** configura um intervalo que incrementa o contador de segundos a cada segundo. A função de limpeza (**clearInterval**) é chamada quando o componente é desmontado.
- O temporizador é configurado na montagem e desmontado quando o componente deixa de existir.

3. Gerenciamento de eventos no React

O **gerenciamento de eventos** em React é semelhante ao gerenciamento de eventos no DOM tradicional, mas com algumas diferenças importantes para tornar o processo mais eficiente e consistente. No React, os eventos são manipulados por meio de funções chamadas **event handlers** que são associadas aos elementos no JSX. O React normaliza esses eventos para garantir que eles funcionem de maneira idêntica em todos os navegadores, oferecendo uma abordagem mais simples e organizada.

3.1 Adicionando Event Handlers

Em vez de usar [addEventListener](#) como no JavaScript puro, no React, os eventos são adicionados diretamente aos elementos como propriedades JSX. A sintaxe é muito parecida com os atributos HTML, mas os nomes dos eventos no React seguem o padrão camelCase (por exemplo, **onClick** em vez de **onclick**).

Veja no quadro a seguir um exemplo simples com o evento **onClick**:

```
function Botao() {  
  const handleClick = () => {  
    alert('Botão clicado!');  
  };  
  return (  
    <button onClick={handleClick}>Clique aqui</button>  
  );  
}  
export default Botao;
```

Neste exemplo, **onClick={handleClick}** associa a função **handleClick** ao evento de clique do botão. Quando o botão é clicado, a função é executada e a mensagem 'Botão clicado!' é exibida.

3.2 Passando Argumentos para Event Handlers

Às vezes, é necessário passar parâmetros para os event handlers. Para isso, usamos **funções anônimas** ou a função [bind](#) para garantir que os parâmetros sejam passados corretamente.

Veja no quadro abaixo um exemplo passando argumentos para o **event handler**:

```
function ListaDeTarefas() {  
  const handleRemover = (id) => {  
    alert(`Remover tarefa com id: ${id}`);  
  };  
  return (  
    <ul>  
      <li>  
        Tarefa 1 <button onClick={() => handleRemover(1)}>Remover</button>  
      </li>  
      <li>  
        Tarefa 2 <button onClick={() => handleRemover(2)}>Remover</button>  
      </li>  
    </ul>  
  );  
}  
export default ListaDeTarefas;
```

Neste exemplo, em **onClick={() => handleRemover(1)}** usamos uma função anônima para passar o parâmetro **1** para a função **handleRemover**. Isso garante que a função seja chamada apenas quando o botão for clicado.

3.3 Eventos Mais Comuns no React

O React suporta uma ampla gama de eventos, incluindo os eventos de formulário, teclado, mouse e outros eventos de interação com o usuário. Veja alguns exemplos abaixo:

Tipo de Evento	Exemplo de Evento	Descrição
Mouse	onClick, onMouseOver, onDoubleClick, onMouseEnter, onMouseLeave	Eventos relacionados a cliques e movimentação do mouse.
Teclado	onKeyDown, onKeyUp, onKeyPress	Captura eventos de pressionamento de teclas.
Formulário	onChange, onSubmit, onFocus, onBlur	Eventos para controle de formulários (campos de texto, seleção, etc.).
Foco	onFocus, onBlur	Eventos de foco em elementos interativos, como inputs.

3.4 Prevenindo o Comportamento Padrão

Assim como no JavaScript tradicional, alguns eventos podem ter comportamentos padrão que você deseja evitar, como o envio de formulários ao clicar em um botão de submit. Para evitar isso, usamos o método **preventDefault()**.

Veja no quadro abaixo um exemplo para prevenir o envio de um formulário:

```
import { useState } from "react";

function FormularioComEnvio() {
  const [nome, setNome] = useState("");

  const handleSubmit = (e) => {
    e.preventDefault(); // Previne o comportamento padrão do formulário
    alert(`Formulário enviado com o nome: ${nome}`);
  };

  return (
    <form onSubmit={handleSubmit}>
      <label>
        Nome:
        <input
          type="text"
          value={nome}
          onChange={(e) => setNome(e.target.value)}
        />
      </label>
      <button type="submit">Enviar</button>
    </form>
  );
}

export default FormularioComEnvio;
```

Neste exemplo, **e.preventDefault()** previne o comportamento padrão do envio do formulário, evitando que a página seja recarregada, mas mostrando a mensagem de **alert()**.

4. Comunicando componentes

A comunicação entre componentes no React é uma parte fundamental para criar aplicações interativas. Como o React segue uma estrutura de componentes em árvore, a comunicação entre eles é feita principalmente por meio de **props** e, em casos mais complexos, por **estado global**. Essa comunicação permite o compartilhamento de estados e funções entre componentes, seja de pai para filho, filho para pai, ou mesmo entre componentes irmãos.

4.1 Comunicando de Pai para Filho (via props)

A comunicação de um componente pai para um componente filho é feita de maneira simples passando **props** (propriedades). Isso permite ao componente filho acessar os dados e funções fornecidos pelo pai.

Veja no quadro abaixo um exemplo de comunicação de pai para filho:

```
// Componente filho
function Saudacao({ nome }) {
  return <h1>Olá, {nome}!</h1>;
}

// Componente pai
function App() {
  const nome = 'Maria';
  return (
    <div>
      <Saudacao nome={nome} />
    </div>
  );
}

export default App;
```

Neste exemplo:

- O componente **App** é o pai, e ele passa o estado **nome** como uma **prop** para o componente filho **Saudacao**.
- No componente **Saudacao**, a prop **nome** é usada para exibir a saudação.

Essa é a maneira mais comum e direta de compartilhar dados de um componente pai para um filho no React.

4.2 Comunicando de Filho para Pai (via props e callbacks)

Para comunicar de um componente filho para um componente pai, normalmente usamos funções **callback**. O componente pai define uma função e a passa para o filho via **props**. O filho pode chamar essa função para enviar dados de volta ao pai.

Veja no quadro abaixo um exemplo de comunicação de Filho para Pai:

```
import { useState } from "react";
// Componente filho
function EntradaNome({ onNomeChange }) {
  const handleChange = (e) => {
    onNomeChange(e.target.value); // Envia o valor digitado de volta ao componente pai
  };
  return (
    <input type="text" placeholder="Digite seu nome" onChange={handleChange} />
  );
}
// Componente pai
function App() {
  const [nome, setNome] = useState("");
  return (
    <div>
      <EntradaNome onNomeChange={setNome} />
      <p>Nome digitado: {nome}</p>
    </div>
  );
}
export default App;
```

Neste exemplo:

- O componente **App** define o estado **nome** e passa a função **setNome** como prop (**onNomeChange**) para o componente filho **EntradaNome**.
- Quando o usuário digita algo no campo de entrada, o componente filho chama a função **onNomeChange** com o valor digitado, permitindo que o componente pai atualize seu estado com o novo valor.

Esse padrão de comunicação de filho para pai é muito útil quando o componente pai precisa responder a eventos gerados pelo filho.

4.3 Comunicando entre Componentes Irmãos (via Pai)

No React, os componentes irmãos não podem se comunicar diretamente. Para compartilhar dados entre componentes irmãos, normalmente o componente pai gerencia o estado compartilhado e passa as informações necessárias para os filhos via **props**. Veja no quadro abaixo um exemplo de comunicação entre Irmãos:

```
import { useState } from "react";
// Componente irmão 1 - Entrada
function Entrada({ onTextChange }) {
  return (
    <div>
      <input
        type="text"
        placeholder="Digite algo"
        onChange={(e) => onTextChange(e.target.value)}
      />
    </div>
  );
}
// Componente irmão 2 - Exibir
function Exibir({ texto }) {
  return (
    <div>
      <h2>Você digitou: {texto}</h2>
    </div>
  );
}
// Componente Pai - App
function App() {
  const [texto, setTexto] = useState(""); // Estado compartilhado
  return (
    <div>
      <h1>Comunicação entre Componentes Irmãos</h1>
      { /* Componente irmão 1 - Envia função para atualizar o texto */ }
      <Entrada onTextChange={setTexto} />
      { /* Componente irmão 2 - Exibe o texto */ }
      <Exibir texto={texto} />
    </div>
  );
}
export default App;
```

Neste exemplo:

- O componente pai **App** gerencia o estado **texto** e passa a função **setTexto** para o componente filho **Entrada** e o valor de **texto** para o componente filho **Exibir**.
- Quando o usuário digita no campo de entrada, o componente **Entrada** envia o valor para o pai, que atualiza o estado e repassa o valor para o componente **Exibir**.

Essa abordagem é comum quando dois ou mais componentes irmãos precisam compartilhar dados ou responder a ações uns dos outros.

Referências

O material desenvolvido para este capítulo baseou-se nas seguintes obras:

- DEVLIN BASILAN DULDULAO, D. B.; CABAGNOT, R. J. L. Practical Enterprise React: Become an Effective React Developer in Your Team. USA: APRESS, 2021.
- ELROM, E. React and Libraries: Your Complete Guide to the React Ecosystem. USA: APRESS, 2021.
- MDN WEB DOCS. Getting started with React. Disponível em: https://developer.mozilla.org/en-US/docs/Learn/Tools_and_testing/Client-side_JavaScript_frameworks/React_getting_started. Acessado em 21 jun. 2024.
- REACT DOCS. Disponível em <https://pt-br.react.dev/>. Acessado em 15 jun. 2024.
- ROLDÁN, C. S. React 17: Design Patterns and Best Practices. UK: Packt Publishing, 2021.
- SCOTT, A. D. JavaScript Everywhere. USA: O'Reilly Media, 2020.
- UZAYR, S. B.; CLOUD, N.; AMBLER, T. JavaScript Frameworks for Modern Web Development: The Essential Frameworks, Libraries, and Tools to Learn Right Now. USA: APRESS, 2019.
- W3SCHOOLS. React Tutorial. Disponível em: <https://www.w3schools.com/react>. Acessado em 17 jun. 2024.